

Discussion 6

A2 Recap & A3 Intro & Routing Data Plane

Outline

- A2 Recap
- A3 Overview
- Routing Data Plane

A2 Recap

- Project 2 Over!
- New project for this semester
 - We wanted to provide an opportunity to build something that you might build in a real job
 - Servicing clients, thinking about how to handle errors, etc.
- Please fill out survey below about what you thought!



Assignment 3

- Project 3 Out!
- Due March 31
- Build reliable transport on top of UDP
- Autograder out soon™ (promise)
- UDP exercise at the end of discussion for you to get familiar with the project

Data Plane

- **Control Plane**
 - Route/Control Processor
 - Responsible for routing protocols, management
- **Data Plane**
 - Linecards (input and output)
 - Interconnect/Switching Fabric
 - Responsible for forwarding packets

- Main tasks:
 - Receive incoming packets (physical layer processing)
 - Update the IP header (TTL, Checksum, Options, Fragment)
 - Lookup the output port for the destination IP address
 - Queue the packet at the switch fabric
- **Challenge:** Processing speed!
 - 100B packets @ 200Gbps → new packet every 4 nanoseconds
 - Typically implemented with specialized ASICs (network processors)

- Longest Prefix Matching (LPM)
 - One entry for each address prefix (not each individual address)
 - For scalability, addresses are aggregated
 - Each prefix is associated with an output port
- **Simple linear search** scales poorly: $O(\text{number of entries})$
- **Tree-based structures** are used for efficient lookups

Output Linecards

- Three key components:
 - **Packet classification:** Map packets to flows
 - **Buffer management:** Decide when and which packet to drop
 - **Scheduler:** Decide when and which packet to transmit

- Classify packets based on multiple header fields:
 - Source/destination IP address (32 bits)
 - Source/destination TCP port number (16 bits)
 - Type of service (TOS) byte (8 bits)
 - Protocol type (8 bits)
- Fields typically specified by range
- Results in a multi-dimensional range search!

- **FIFO (First-In-First-Out):**
 - Simplest approach
 - No classification, drop-tail buffer management
 - Packets scheduled in arrival order
- **Priority Scheduling:**
 - Packets in higher priority queues served first
 - Lower priority queues served only when higher priority queues empty
- **Round-Robin:**
 - Serve packets from each queue in turn
 - Fair allocation of bandwidth

- Extends round-robin for variable-sized packets
- For each packet, compute the time when its last bit would leave the router if flows are served bit-by-bit
- Serve packets in increasing order of these deadlines
- Results in $\sim$ min-max fair allocation of bandwidth across flows

Weighted Fair Queuing (WFQ)

- Assign different flows different shares of bandwidth
- Extension of Fair Queuing
- Today implemented in almost all routers
- Primarily used to isolate traffic at larger granularities (e.g., per-prefix)

- Given set of bandwidth demands r_i and total bandwidth C :
 - $a_i = \min(f, r_i)$
 - where f is the unique value such that $\sum a_i = C$
- **Key property:** If you don't get your full demand, no one gets more than you

Example

$C = 10$ Gbps; $r_1 = 8$ Gbps, $r_2 = 6$ Gbps, $r_3 = 2$ Gbps

Calculate the Min-Max fair allocation.

Example

$C = 10$ Gbps; $r_1 = 8$ Gbps, $r_2 = 6$ Gbps, $r_3 = 2$ Gbps

Calculate the Min-Max fair allocation.

Answer

1. Start with equal share: $\frac{C}{3} = 3.33$ Gbps
2. Flow 3 only needs 2 Gbps, so it gets 2 Gbps
3. Remaining capacity: $C - 2 = 8$ Gbps for 2 flows
4. New equal share: $\frac{8}{2} = 4$ Gbps
5. Since both r_1 and r_2 want more than 4 Gbps, they each get 4 Gbps

Final allocation: $a_1 = 4$ Gbps, $a_2 = 4$ Gbps, $a_3 = 2$ Gbps

Interactive Exercise: UDP

So far, we've learned about UDP conceptually. Let's take a look at how we use UDP in C++.

Before we start, there is one very important distinction to make from TCP: UDP is connectionless and operates using datagrams (packets). This means that there is no bytestream abstraction.

Important: This means that when you say

“I want to receive n bytes”

You will receive at most n bytes from the **current** packet. If the packet was larger than n bytes, the extra bytes will be lost!

Losing Bytes: Illustrative Example

Consider the following psuedocode and its result using TCP/UDP:

```
1  send(ABC)
2  send(DEFG)
3  ---
4  recv(2)
5  recv(3)
```

TCP

```
1  recv(2) -> AB
2  recv(3) -> CDE
```

UDP

```
1  recvfrom(2) -> AB
2  recvfrom(3) -> DEF
3  // C was lost!
```

To create a UDP socket, we can use the `socket` function in C++.

```
1 int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

`SOCK_DGRAM` indicates that we are creating a UDP socket.

To bind a socket to a port, we can use the `bind` function in C++.

```
1 bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
```

- This binds the socket to the counterparty information given in `server_addr`
- Remember that UDP is connectionless, so there's no equivalent to `listen` or `connect` from TCP

To receive a packet, we can use the `recvfrom` function in C++.

```
1  recvfrom(sockfd, buffer, sizeof(buffer), flags, (struct sockaddr  
   *) &client_addr, &client_len);
```

- Note that there are two extra parameters compared to `recv`:
 - `client_addr`: A pointer to a `sockaddr` structure that will store the address of the sender
 - `client_len`: The length of the `client_addr` structure
- This is because UDP is connectionless, so we need some other way to identify who the sender is

To send a packet, we can use the `sendto` function in C++.

```
1 sendto(sockfd, buffer, sizeof(buffer), flags, (struct sockaddr  
  *)&client_addr, sizeof(client_addr));
```

- This is similar to `recvfrom`, but instead of receiving a packet, we are sending a packet
- The `client_addr` and `client_len` parameters are the same as in `recvfrom`, but are used to identify the receiver

Aside: What is `std::expected`?

Shortly, it's an alternative to throwing exceptions.

It's useful to write functions that can return either a successful value or an error. For example:

```
1 std::expected<int, std::string> divide(int a, int b) {  
2     if (b == 0) {  
3         return std::unexpected("Division by zero");  
4     }  
5     return a / b;  
6 }
```

```
1  int main() {
2      std::expected<int, std::string> result = divide(123, 0);
3
4      if (result) {
5          std::cout << "Parsed value: " << *result << '\n';
6      } else {
7          std::cout << "Error: " << result.error() << '\n';
8      }
9
10     return 0;
11 }
```

Exercise: Game Server and Client

Suppose that you are building a multiplayer game “489: The Game.”

In this game, the server periodically sends state updates to each client over UDP for each player. There are two types of updates: `PlayerUpdate` and `GameUpdate`, shown below.

PlayerUpdate Packet	
Attribute	Type
Type	UpdateType update_type;
Player ID	int id;
Position	struct Position { int x; int y; };
Health left	int health;

GameUpdate Packet	
Attribute	Type
Type	UpdateType update_type;
Game ID	int id;
Time Left	int time_left;

Exercise: Game Server and Client

Your task is to implement both the `GameServer` and `GameClient` classes, along with some utility functions to help you use UDP more easily.